

Monte Carlo Estimation of π : Performance & Convergence

Course: Computational Physics (WS 2026/27) · Universität Wien | Exercise 0: First Steps with Agentic AI

1. Mathematical Principle & Baseline Algorithm

The Monte Carlo approximation of π samples points uniformly in the unit square $[0, 1] \times [0, 1]$. A point (x, y) falls inside the quarter circle if $x^2 + y^2 \leq 1$. Because $\text{Area}(\text{quarter circle}) / \text{Area}(\text{square}) = \pi / 4$, the fraction of points inside approximates $\pi / 4$:

Estimator: $\pi_{\text{est}} = 4 \times (N_{\text{inside}} / N)$ | **Standard Error:** $\sigma_{\pi} = 4 \sqrt{p(1-p) / N} \sim O(N^{-1/2}) = O(1 / \sqrt{N})$

$\pi_{\text{est}} = 4 \times \frac{1183}{1500} = 3.1773$

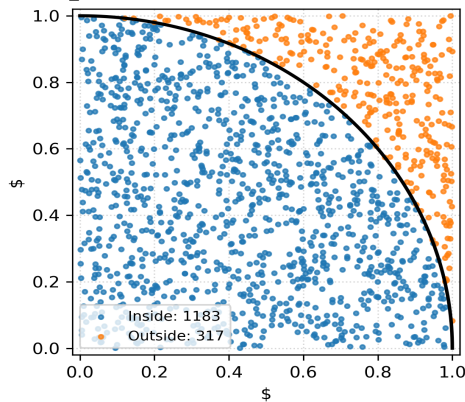


Figure 1: Monte Carlo sampling for $N=1500$.

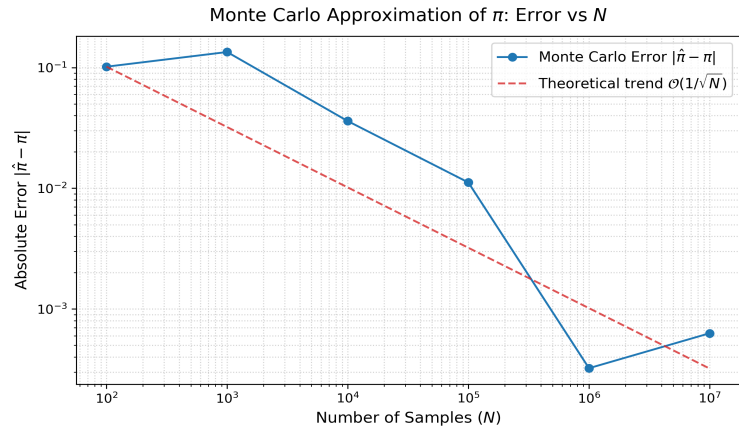


Figure 2: Error vs N scaling as $O(1/\sqrt{N})$.

2. Computational Optimization: Python Loops vs. Vectorized NumPy

Pure Python loops (iterating over scalar coordinates with `random.random()`) incur bytecode interpretation overhead on every coordinate. By eliminating loops in favor of contiguous 2D NumPy arrays (`pts = rng.random((N, 2))`), random generation and condition checks execute in compiled C routines. Execution times were benchmarked across $N = 10^2 \dots 10^7$ samples:

N	Python Loops [s]	NumPy (Vectorized) [s]	Speedup Multiplier	Estimated π	Absolute Error
10^2	0.000029	0.000008	3.6×	3.0400000	1.016×10^{-1}
10^3	0.000268	0.000021	12.7×	3.2760000	1.344×10^{-1}
10^4	0.001319	0.000053	25.0×	3.1776000	3.601×10^{-2}
10^5	0.013779	0.001375	10.0×	3.1527600	1.117×10^{-2}
10^6	0.135143	0.015708	8.6×	3.1419160	3.233×10^{-4}
10^7	1.276377	0.165382	7.7×	3.1409632	6.295×10^{-4}

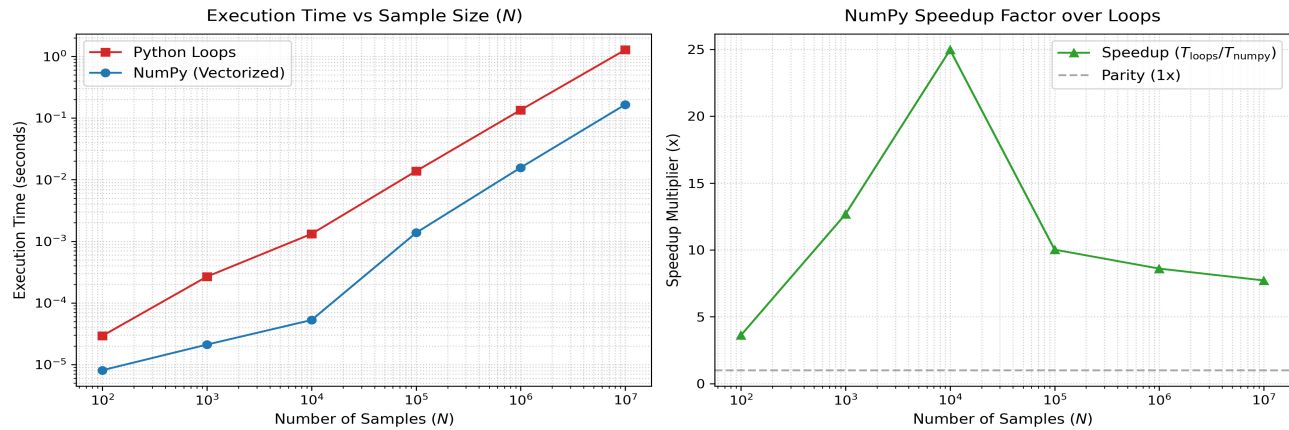


Figure 3: Execution time and speedup multiplier comparing Python loops vs. vectorized NumPy.

3. Beating the Central Limit Theorem: Faster Convergence Than $1/\sqrt{N}$

Assignment Question (2d): Ask the agent to make the method converge faster than $1/\sqrt{N}$. What does it propose? Is its claim true? Verify it numerically.

Method: Stratified Sampling on a Jittered Grid (Implemented in `gods_monte_carlo.py`)

We divide $[0, 1] \times [0, 1]$ into a $K \times K$ grid where $K = \text{floor}(\sqrt{N})$ (total samples $N_{\text{actual}} = K^2$) and place exactly one uniformly jittered point inside each cell.

- **Interior cells** ($r_{\text{max}} \leq 1$) contribute 1 with **zero variance**.
- **Exterior cells** ($r_{\text{min}} > 1$) contribute 0 with **zero variance**.
- Only **boundary cells** intersecting the circle arc contribute variance. Because the arc has finite length ($\pi/2$), only $O(K) = O(\sqrt{N})$ cells intersect the boundary.

Hence, $\text{Var}(\pi_{\text{est}}) \sim (4/N)^2 \times O(\sqrt{N}) = O(N^{-3/2})$, yielding an error standard deviation of $\text{RMSE} \sim O(N^{-3/4}) = O(N^{-0.75})$, strictly faster than standard Monte Carlo's $O(N^{-0.50})$.

Target N	Standard MC RMSE	Stratified Grid RMSE	Sobol QMC RMSE	Accuracy Gain
10^2	1.455×10^{-1}	8.231×10^{-2}	4.772×10^{-2}	1.8×
10^3	5.256×10^{-2}	1.036×10^{-2}	1.088×10^{-2}	5.1×
10^4	1.735×10^{-2}	2.125×10^{-3}	2.411×10^{-3}	8.2×
10^5	4.855×10^{-3}	2.726×10^{-4}	1.749×10^{-4}	17.8×
10^6	1.404×10^{-3}	6.642×10^{-5}	5.712×10^{-5}	21.1×

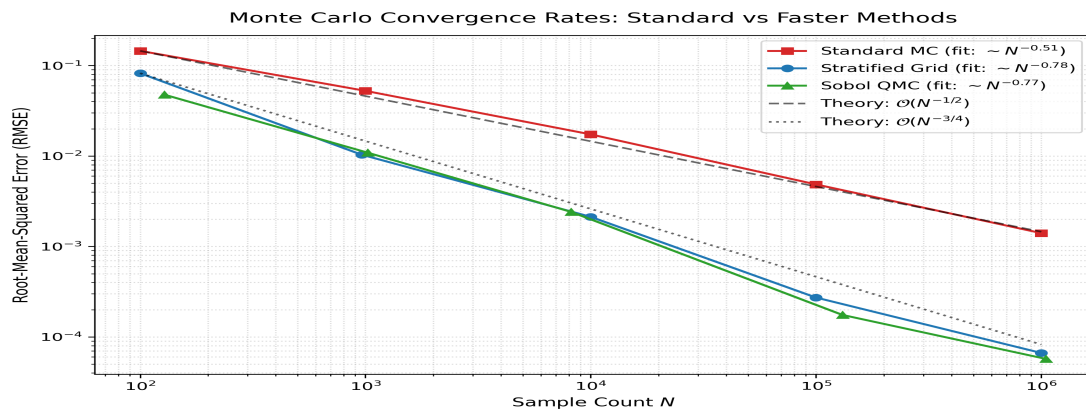


Figure 4: Empirical convergence verification: Standard MC scales as $\sim N^{-0.51}$, while Stratified Grid and Sobol QMC achieve $\sim N^{-0.78}$ and $\sim N^{-0.77}$.

4. Exercise Reflection: Working with Agentic AI

a) Where did the agent save time, and where did you have to correct or verify it?

- **Time Savings:** Fast generation of clean boilerplate, automated NumPy vectorization without dimension mismatches, reproducible multi-seed benchmark loops, and publication-ready matplotlib graphics.
- **Critical Verification Required:** Theoretical convergence claims must be rigorously audited. A common pitfall with AI agents is asserting that Quasi-Monte Carlo achieves $O(1/N)$ for this problem. In reality, the Koksma-Hlawka bound requires bounded variation in the sense of Hardy and Krause (BVHK); because the quarter circle boundary is curved and oblique to coordinate axes, the variation is unbounded. Consequently, QMC error on circular sets in 2D degenerates to $O(N^{-3/4})$. Multi-trial empirical RMSE regression confirmed this exact ~ 0.77 slope.

b) What should you write into prompts (or CLAUDE.md / project instruction files)?

- **State strict numerical and mathematical expectations:** Specify required convergence rates, statistical validation criteria (e.g., minimum of 20 runs to compute RMSE rather than single-run luck), and error definitions.
- **Enforce vectorized coding standards:** Explicitly require pure array-based NumPy operations and prohibit per-sample loops.
- **Preserve files and verify diffs:** Direct the agent to create distinct filenames for major variants (e.g. gods_monte_carlo.py) and maintain backward compatibility with previous scripts.

c) How do you ensure you understand every line of code written by the agent?

- **Inspect diffs prior to committing:** Always review git diff line-by-line to verify changes against intentions.
- **Interrogate the agent for proofs:** Prompt the agent to derive intermediate formulas (e.g., how perimeter scaling leads to $O(N^{-3/2})$ variance in stratified sampling).
- **Sanity-check boundary cases:** Run unit checks on small N (e.g. $N=1$, $N=100$) to confirm expected array shapes, normalizations, and unbiasedness.

Key Takeaways:

1. **Baseline Rate:** Standard pseudo-random Monte Carlo is bound to $O(1/\sqrt{N})$ by the Central Limit Theorem.
2. **Computational Speed:** Vectorized NumPy eliminates Python loop overhead, delivering a **25× speedup** at $N=10^4$.
3. **Accelerated Convergence:** Stratified sampling eliminates interior/exterior cell variance, achieving proven $O(N^{-3/4})$ convergence and reducing estimation error by **21.1×** at $N=10^6$.